

scripts/verify-all-constitution-rules.sh — User Guide

Last verified: 2026-08-25 (gate-execution isolation; §6.N.2 gate-shaping bluff hunt of 2026-08-23) **Inheritance:** HelixConstitution §11.4.32 (Post-Constitution-Pull Validation Mandate) + §11.4.18 (script documentation)

Overview

Master enforcement-engine sweep that wraps every individual constitution-gate + every hermetic test suite into a single invocation. Per §11.4.32 itself: "this gate is the enforcement engine for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced in the codebase".

Mandatory invocation triggers (per §11.4.32):

- Whenever git submodule update --remote constitution produces any content change in the constitution submodule
- Operator-explicit manual invocation
- ci.sh step 5a4 (every CI run)

Usage

```
bash scripts/verify-all-constitution-rules.sh #
    STRICT (default) – exits 1 on any gate failure
bash scripts/verify-all-constitution-rules.sh --strict #
    explicit strict
bash scripts/verify-all-constitution-rules.sh --advisory #
    exits 0 even with failures
bash scripts/verify-all-constitution-rules.sh --json-only #
    suppresses stdout summary; only emits JSON
```

Inputs

Arg	Description
--strict	Default: exit 1 on any gate failure
--advisory	

Arg	Description
	Exit 0 even with failures (for incremental adoption)
<code>--json-only</code>	Suppress stdout per-gate summary; only write attestation JSON
<code>LAVA_REPO_ROOT</code> <code>env</code>	Override repo root for hermetic testing

Outputs

- **stdout:** per-gate pass/fail summary + final verdict (suppressed by `--json-only`)
- **.lava-ci-evidence/verify-all/<UTC-timestamp>.json:** structured attestation with sweep metadata + per-gate result list

Attestation JSON schema:

```
{
  "sweep_timestamp": "<ISO timestamp>",
  "sweep_mode": "strict|advisory",
  "sweep_constitution_pin": "<git rev-parse HEAD inside
    constitution/>",
  "total_gates": <int>,
  "pass_count": <int>,
  "fail_count": <int>,
  "all_passed": <bool>,
  "gates": [
    { "name": "<gate-name>", "rule_ref": "<$clause-ref>",
      "result": "PASS|FAIL", "duration_seconds": <int> }
  ]
}
```

Gates registry (current)

The registry is **part static, part discovered at run time**, so the total is not a constant in the source: a fixed list of scanner rows is followed by loops that walk `tests/pre-push/check*_test.sh` and `tests/check-constitution/test_*.sh` and register one gate per file found. Adding a hermetic test file to either directory therefore adds a gate with no edit to this script.

The most recent measured total is **57 gates**, recorded in `.lava-ci-evidence/verify-all/2026-08-23T09-47-52Z.json` (`total_gates: 57`). Treat any number written here as a snapshot with a date attached, never as an invariant — read `total_gates` out of the newest attestation for the current figure. The four categories:

1. **Constitution doc parser** (1 gate) — `scripts/check-constitution.sh` covering §6.A-§6.AE inheritance + §6.W boundary + §11.4.6 no-guessing
2. **Anti-bluff scanners** (9 gates) — non-fatal coverage / **gitignore coverage (Phase 2 — §11.4.30) / nested-own-org-submodules (Phase 5 — §11.4.28, STRICT) / canonical-root-and-upstreams (Phase 8 — §11.4.35 + §11.4.36, STRICT) / helix-deps-manifest (Phase 3 — §11.4.31, STRICT) / coverage-ledger (Phase 7 — §11.4.25, ADVISORY)** / Challenge discrimination Layer 1+2 / Challenge coverage / fixture freshness
3. **No-hardcoding scanners** (3 gates) — UUID / IPv4 / host:port literal scans
4. **Hermetic test suites** (~23 gates) — `tests/firebase` + `tests/ci-sh` + `tests/compose-layout` + `tests/tag-helper` + `tests/vm-*` + `tests/pre-push/check{4,5,6,7,8,9}` + `tests/check-constitution/*` (now includes `test_gitignore_coverage.sh` + `test_nested_own_org_submodules.sh` + `test_canonical_root_and_upstreams.sh` + `test_helix_deps_manifest.sh` + `test_coverage_ledger.sh`)

Per-gate STRICT/ADVISORY note: Phase 5 / Phase 8 / Phase 3 gates have been STRICT-flipped 2026-05-15 after their respective debt closures landed:

- **Phase 5-debt closure:** github cascade merge removed `Challenges/.gitmodules` (CONST-051(C) flat-layout enforcement); Panoptic no longer nested.
- **Phase 8-debt closure:** 10 `install_upstreams` scripts landed via Phase 8-debt batch (Challenges, Config, Containers, Discovery, HTTP3, Mdns, Middleware, RateLimiter, Recovery, Tracker-SDK each gained `install_upstreams.sh` + `Upstreams/GitHub.sh` + `Upstreams/GitLab.sh` per §6.W).
- **Phase 3-debt closure:** 16/16 per-submodule `helix-deps.yaml` manifests landed via Phase 3-debt batch (each authored with HONEST per-submodule deps after analysis).

Phase 7 STRICT-flipped 2026-05-16 after the waiver-backfill agent (commit 20b3fd36) lifted the ledger from 0/20/38 to 48/10/0. The 10 remaining partial rows are honest gaps for structural/glue modules without dedicated unit tests (transitively exercised by downstream features + Challenges). The verifier hard-asserts row-coverage + freshness + schema integrity in STRICT mode; ledger-staleness conditions are the gate's primary failure mode (legitimate gap counts don't trigger the verifier).

The list grows as new constitution gates land. Adding a *scanner* gate requires editing both the sweep and the hermetic meta-test (tests/check-constitution/test_verify_all_rules.sh); adding a *hermetic test file* under tests/pre-push/ or tests/check-constitution/ requires neither, because the discovery loops pick it up by filename.

Gate-execution isolation — each gate runs in a subshell

run_gate evaluates a gate's command string **inside a subshell**: `if ( eval "$cmd" )`, not `if eval "$cmd"`. The parentheses are the whole mechanism, and they are load-bearing in two distinct ways.

The defect this fixed

Found by the §6.N.2 gate-shaping bluff hunt of 2026-08-23. Every gate's command string used to be evaluated in **the sweep's own shell**. A command string that reached `exit` therefore did not fail one gate — it terminated the entire sweep process from inside the gate. The consequences compound:

- every gate registered *after* the exiting one never ran, and nothing said so;
- the GATE_RESULTS rows already recorded — **including FAIL rows** — were discarded unread, because the arrays live in the shell that just died;
- no attestation JSON was written at all, since the writer runs at the end of the sweep;
- and when the gate's command ended in `exit 0`, the sweep process itself exited **0**.

That last combination is the anti-bluff failure: a sweep that stopped part-way through its registry, dropped recorded failures on the floor, produced no evidence at rest, and still handed its caller a green exit status. §11.4.32 names this shape directly — "*A sweep that exits PASS without running every implementable gate is a §11.4.32 violation.*" The sweep is the enforcement engine for every other gate, so a green-on-truncation sweep silently disarms all of them at once.

The pre-fix source carried a comment acknowledging the hazard and asking gate authors to work around it ("*Use false (or an external command) to signal failure*"). That put the invariant in the hands of every future gate author, on every future row, forever — a convention, not a mechanism. The subshell makes it structural: `exit` inside a gate now ends the subshell, `run_gate` reads its status like any other non-zero exit, and the row is recorded as FAIL.

This is not a theoretical hazard for this particular script. The registry deliberately contains **guard rows whose entire purpose is to fail** — the MISSING or non-executable: <suite>/run_all.sh row, the NO tests/pre-push/check*_test.sh matched row, and the NO tests/check-constitution/ test files matched row, all added by the preceding 2026-08-22 bluff hunt so that a vanished test suite registers as a FAIL instead of silently shrinking the registry. Each is written today as ...; false. Written the more natural way — ...; exit 1 — any one of them would have truncated the sweep at the exact moment it was reporting that a test suite had gone missing.

The second property: a gate cannot rewrite the verdict

Because the command runs in a child shell, an assignment it performs cannot reach the parent. A gate command containing GATE_RESULTS=(PASS) mutated the sweep's own bookkeeping under the old form and could erase an earlier FAIL from the final verdict. Under the subshell it changes only the child's copy, which is discarded.

Regression coverage

tests/check-constitution/test_verify_all_gate_isolation.sh — three cases, all asserting on the **real** run_gate, which the test extracts verbatim out of this script at run time (via awk, anchored from declare -a GATE_NAMES= to the next column-0 }) rather than re-implementing it. A copy-paste re-implementation would keep passing after the shipped function regressed; extraction cannot.

Case	Registry driven	Asserted
test_exiting_gate_does_not_abort_sweep	false → exit 0 → true	all three rows registered (REGISTERED=3) and RESULTS=FAIL PASS PASS — the exit 0 gate is one PASS row, not the end of the sweep REGISTERED=2, RESULTS=FAIL PASS — the shape the guard rows above would
test_nonzero_exiting_gate_records_fail	echo boom >&2; exit 1 → true	

Case	Registry driven	Asserted
test_gate_cannot_mutate_sweep_state	false → GATE_RESULTS=(PASS); true	have had with exit 1 RESULTS=FAIL PASS — the earlier FAIL survives a gate that tried to clobber the array

Each case fails loudly against the pre-fix form: the harness produces no output at all, because the shell it was running in is gone. The test reports that explicitly rather than comparing against an empty string — got: <no output – shell terminated>.

The file is picked up automatically by the tests/check-constitution/test_*.sh discovery loop, so it is a registered gate of the sweep it protects.

Falsifiability rehearsal (mandated by §11.4.32 itself)

§11.4.32: "sweep's own meta-test (paired mutation §1.1) plants a known violation of each enforced gate and asserts sweep reports FAIL for the planted gate."

Implemented in tests/check-constitution/test_verify_all_rules.sh with 4 fixtures:

- test_clean_tree_passes — sweep passes against the real Lava tree
- test_gate_failure_propagates — synthetic minimal repo (no constitution clauses) causes sweep to report all_passed: false + exit 1
- test_advisory_mode_returns_zero — --advisory exits 0 even with failures
- test_attestation_json_structure — emitted JSON has all required fields

A sweep that exits PASS without running every implementable gate is itself a §11.4.32 violation.

Side-effects

- Creates .lava-ci-evidence/verify-all/ directory if missing

- Writes one attestation file per invocation (timestamps make file names unique)
- Does NOT modify any tracked files

Cross-references

- HelixConstitution Constitution.md §11.4.32 (the mandate)
- docs/plans/2026-05-15-constitution-compliance.md (Phase 1 — this script)
- docs/helix-constitution-gates.md CM-VERIFY-ALL-CONSTITUTION-RULES row
- tests/check-constitution/test_verify_all_rules.sh (the hermetic meta-test)
- tests/check-constitution/test_verify_all_gate_isolation.sh (gate-execution isolation)
- .lava-ci-evidence/bluff-hunt/2026-08-23-cycle-gate-shaping-scripts.json (the §6.N.2 hunt that found the truncation defect)
- All individual scanners + hermetic suites the sweep wraps

Two anti-bluff properties of the sweep itself (added 2026-08-22)

A missing test suite is a FAIL row, not a silent skip

Each hermetic suite is registered as a gate by locating its `run_all.sh`. Previously a suite whose `run_all.sh` was deleted, renamed, or had merely lost its executable bit was **skipped silently** — the gate vanished from the registry and the sweep still reported a clean `N PASS / 0 FAIL`, with no signal that `N` had shrunk.

Losing a test suite is precisely the drift this sweep exists to catch, so a missing or non-executable `run_all.sh` now registers as a FAIL row naming the path:

MISSING or non-executable: `<suite>/run_all.sh`

The same reasoning applies to the flat-layout suites: a glob that matches nothing used to register nothing, so an emptied `tests/pre-push/` read as *"no pre-push tests failed"* rather than *"no pre-push tests ran"*. A `prepush_registered` counter now makes the difference visible.

\$cmd runs via eval in THIS shell, not a subshell

Gate command strings are evaluated in the sweep's own shell. A command string ending in `exit N` therefore terminates the **whole sweep** rather than failing one gate — the remaining gates never run, and the sweep's summary reflects only what executed before it.

When authoring a gate command that must signal failure, use `false` (or an external command that exits non-zero), never a bare `exit`.

\$cmd now runs in a SUBSHELL — superseding the note above (2026-08-23)

The section above records that gate command strings were evaluated in the sweep's **own** shell. That is no longer true, and the change is the point.

In the old form, a gate command that reached `exit` terminated the **entire sweep** rather than failing one gate: every later gate was silently skipped, the FAIL rows already recorded were discarded, no attestation was written — and for `exit 0`, the sweep itself exited **0**. §11.4.32 is explicit: *"A sweep that exits PASS without running every implementable gate is a §11.4.32 violation."*

`$cmd` is now evaluated inside a subshell — `if ( eval "$cmd" ) >/dev/null 2>&1; then` — which also prevents a gate from mutating `GATE_RESULTS` / `GATE_NAMES` and rewriting the sweep's own verdict. Covered by `tests/check-constitution/test_verify_all_gate_isolation.sh`.

The advice in the earlier section still stands for gate authors: signal failure with `false` or an external non-zero command, not a bare `exit`. The subshell now contains the blast radius if someone does it anyway.

The suite list is an ARRAY, so the registry floor derives its own expectation

The hermetic-suite list is held as an array rather than being iterated inline, so the registry floor can compute how many suites it *should* have registered from the list itself. A hardcoded count would go stale the moment a suite is added or removed — a stale floor is the same defect as no floor, wearing a number.